

A Complete Windows Guide for Running Tensorflow with GPU Support

Katherine Kaylegian-Starkey^{1, 2} and Ben McMorran¹

¹University of Oregon, 185 E 13th Ave, Eugene, OR 97403

²Thermo Fisher Scientific, 5350 NE Dawson Creek Dr., Hillsboro, OR 97124

July 2025

1 Introduction

Neural networks, which take the form of classifiers and regressors, are at the root of all AI. The former allows us to separate components while the latter makes educated guesses at what comes next. Using these tools, we have embarked on a new era where computers are as much companions as dogs and where algorithms work equally as hard as researchers to solve complex computational problems. This new computing power has allowed the world to excel in every field from pharmacology to particle physics. Regardless of the type of neural network or the problem it solves, training complex AI models is a computationally expensive task that necessitates specific hardware and software requirements. While most hardware is agnostic, Nvidia has become a favorite for graphics processors (GPUs) while Keras and Tensorflow have become open-sourced and easily accessible favorites for building models. This combination of hardware and software does not communicate innately, and the crux of model training lies in the ability to build a system capable of handling the training requirements.

Optimized GPU-enabled model training using Tensorflow and Keras results from a delicate combination of hardware and software. While Mac operating systems typically alleviate many of the difficulties encountered when running these programs through Windows, corporate requirements often insist on Windows-based operating systems. Troubles with Nvidia communicating with Windows communicating with an IDE and finally communicating with Tensorflow are not new, and endless blogs and forums exist describing how to perform seamless installs and debugs of these programs. Unfortunately, the plethora of information on the web can be confusing, circuitous, and often does not provide relief from the given problem. This guide aims to eliminate the mystery of the software and hardware communication required for GPU-enabled model training with Tensorflow and Keras by providing detailed instructions for installations and

explanations for troubleshooting a wide range of problems that may be encountered while setting up a GPU-enabled Windows/Tensorflow system or while training models on a GPU-enabled Windows/Tensorflow system.

If possible, we recommend running a pure or dual-booted Linux box to ensure a stable training environment instead of training using Windows Subsystem for Linux (WSL) in a Windows Native environment. If that isn't possible or if a WSL install is desired, this guide will shed light on the web of requirements necessary for efficient CUDA and Tensorflow installations and GPU-enabled model training on a Windows Native system. The steps that follow won't work for everyone, but hopefully the authors can share enough knowledge to make further troubleshooting painless. We begin with a brief dedication and an overview of hardware and software requirements. This is followed by a detailed installation guide. Next, we provide a comprehensive troubleshooting guide and conclude with our final thoughts. If you are reading this, you are not alone.

2 Dedication

This guide is dedicated to the endless hours spent repeatedly installing, wiping, and reinstalling everything Nvidia, CUDA, and Tensorflow after required company updates broke Tensorflow and GPU communication and the endless hours attempting installs when this communication broke for no reason while training models.

3 Hardware and Software Requirements

3.1 Hardware

Before installing software, all hardware requirements must be satisfied, and the following hardware is necessary for GPU-enabled model training:

1. A base PC running Windows
2. A GPU
3. An external or internal hard drive to hold data
4. Extra RAM, although this isn't strictly necessary.

We are building the system from the ground up, so a mid-range tower running a new version of Windows is sufficient. The authors are using a Dell Precision 5680 Tower running Windows 11 Enterprise on a 64-bit operating system; this information can be found by going to the Search bar and typing Settings then navigating to System → About. This system information will be important when choosing drivers. The most commonly used GPUs are made by Nvidia, and many computers come with preinstalled Nvidia cards. Because Nvidia GPUs are the most common, from this point on, this guide references Nvidia cards and drivers. Our system came with a RTX Ada Gen 2000 (A2000)

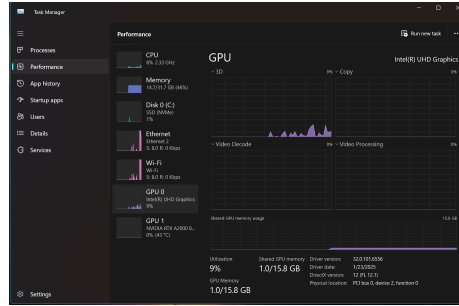


Figure 1: Windows Task Manager shows that this notebook has two GPUs installed; one is a Nvidia A2000.

card. This is sufficient for small models and small datasets. For computationally expensive models running large datasets, a card with more dedicated GPU memory and computational power is required. Detailed information about Nvidia GPUs can be found at <https://www.nvidia.com/en-us/geforce/graphics-cards/>. We added a RTX Ada Gen 5000 (A5000) card while keeping the A2000 card for dual-GPU model training. Once complete, the success of the GPU installation can be validated by confirming that the system sees the new device in the Windows Task Manager. To open the Task Manager, navigate to the Search bar and type *Task*; a list of apps will appear on the left of the screen. Click on Task Manager. Once open, a list of tabs appears on the left; click on the Performance tab. This will show all the components that are visible to the tower. Ensure that the newly installed GPU is listed near the bottom, **Fig. 1**.

Once the base system is built it's important to have an external or internal hard drive capable of housing data. Windows syncs to OneDrive which doesn't have enough memory to hold any more than the equivalent of about 3000 512x422 .tiff images. Windows will save data in the cloud and transfer it to the PC only when Tensorflow calls for it; this can create data loading bottlenecks before training even begins, see **5.3.1**. The easiest way to avoid this slowdown is to have a 1 terabyte (TB) or larger hard drive specifically for holding data; the authors have both a 4 TB external and a 6 TB internal hard drive. External hard drives are connected through the USB port and require configuration before WSL can read them. Internal hard drives can be installed following the base PC's instruction manual; a quick internet search with the keywords *install second hard drive <computer make and model>* is also sufficient.

As noted above, extra RAM is not strictly necessary, but it may help alleviate the data loading bottlenecks mentioned above. If additional RAM is desired, use the PC's manual to find the specifications of the already installed memory card; the additional RAM must match the specifications of original RAM. Note the manufacturer, speed in Hertz (Hz), memory size in gigabytes (GB), and the data rate such as DD4 or DDR5. The new memory can be installed using the PC's manual or an internet keyword search. If the system doesn't boot or

doesn't remain booted after RAM (or GPU) installation see **5.1.2.1 (5.1.1.2)**.

3.2 Software

A correct software installation requires installing compatible distributions, and incompatibilities can cause data loading and model training issues such as improper GPU use, no GPU use, or slowdowns and freezes, see **5.3.2**. It's also crucial that programs are installed in order so that each dependency can see its parent, although an IDE can be installed at any point after step 3. Below is a list of required software; installation details are in **4**.

1. A graphics driver
2. WSL
3. Python *a default version is installed with WSL but additional versions may be installed if desired
4. CUDA
5. cuDNN
6. An IDE
7. Tensorflow.

4 Software Installation and Validation

4.1 Graphics Drivers

GPUs provide heightened graphic and computational processing for gaming and scientific computation but are not stand-alone components; they require card and system-specific drivers to interact with the PC. Using the system information found in **3.1**, navigate to <https://www.nvidia.com/en-us/drivers/> and choose the newest appropriate driver. Download the Windows Native executable, and run the installer once the download is complete. When asked to perform an Express or Custom install, choose Custom. All included packages should be checked as well as the box for a clean installation which ensures that all previous settings are cleared. Once the installation is complete restart the computer, open a command prompt, and type *nvidia-smi*. This command displays the currently installed Nvidia driver and can be used to validate the driver installation. The output of this command after a successful installation is pictured in **Fig. 2**; if *nvidia-smi* returns nothing or only shows a subset of installed GPUs see **5.2.1.1**.

```
Wed Jul 23 14:00:33 2020
```

NVIDIA-SMI 576.57.04				Driver Version: 576.52		CUDA Version: 12.9	
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile Uncorr. ECC		
Fan	Temp	Perf	Power Usage/Cap	Memory-Usage	GPU-Util	Compute M.	NIC M.
0	NVIDIA RTX A3000 12GB	On	00000000:55:00:0	On	0	0	0
30%	48C	P8	12W / 70W	200MiB / 11510MiB	4%	Default	N/A
1	NVIDIA RTX A5000	On	00000000:A2:00:0	Off	0	0	0
30%	27C	P8	6W / 230W	90MiB / 23020MiB	0%	Default	N/A

Figure 2: After successfully installing a graphics driver, the output of *nvidia-smi* will show the installed driver, the CUDA version it was compiled with, and all of the installed GPUs supported by the driver.

4.2 WSL

In a command prompt type *wsl.exe --install -d <Distribution Name>*. Available distributions, such as Ubuntu, can be found by running *wsl --list --online* [1] and are a matter of preference. Our installation was successful by running *wsl.exe --install -d Ubuntu*. Installing WSL using *wsl --install -d <Distribution Name>* as shown in [2] may work, but this method may cause the command *wsl* to not open the installed distribution, so it is recommended to use the *wsl.exe* installation command. Once the installer finishes, type *wsl* in the command prompt. The installed distribution will open and request that a user name and admin password be set. Follow the prompts to complete the installation.

4.3 Python

WSL installs a default version of Python that can be queried from the WSL prompt using *pythonv -V*. At publication, the current Python version is Python3, so the command is *python3 -V*. This should output something like Python 3.v.v; for the authors this looks like Python 3.10.12. If the output is blank see **5.2.3**. If a version other than the default is desired, open the WSL prompt and type:

1. \$ sudo apt-get update
2. \$ sudo apt-get upgrade
3. \$ sudo apt install pythonv.v

The first two commands update the Linux distribution while the third installs the new version of Python. If the installed Linux distribution doesn't use the *sudo* command a quick internet query can show how install it for the given distribution. To validate the installation, navigate to */usr/bin* in the WSL prompt. If the installation was successful, files for the root and new Python version will be present.

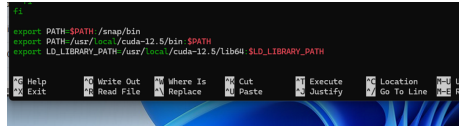


Figure 3: An example of adding the CUDA path to environmental variables in the bashrc file.

4.4 CUDA

CUDA stands for Compute Unified Device Architecture and facilitates data transfer between the system’s CPU and GPU [3]. Before starting the initial CUDA installation, check <https://www.tensorflow.org/install/source#gpu> to find the CUDA version required for the newest Tensorflow version. Unless an older Tensorflow distribution is specifically required, it’s best to install the newest version.

Just like the previously installed Nvidia driver was compiled with a CUDA toolkit distribution, CUDA toolkit installers are compiled with a driver. To ensure that the newest GPU driver is being used and that the proper CUDA version is being accessed, install CUDA through WSL, not Windows Native; this installation method guarantees that CUDA will be installed into `/usr/local/bin` and that the graphics driver installed in 4.1 will be used. From the CUDA archive, <https://developer.nvidia.com/cuda-toolkit-archive>, choose the CUDA toolkit installer that matches the operating system and Tensorflow version dependency. Follow the directions for the WSL deb local installation from the Nvidia archive then add

`export PATH=/usr/local/cuda-v.v/bin:$PATH` and
`export LD_LIBRARY_PATH=/usr/local/cuda-v.v/lib64:$LD_LIBRARY_PATH`
to the bashrc file. For example, using the nano text editor, this can be accomplished with the command `nano ~/.bashrc`. Scroll to the bottom of the file, copy both `echo` commands above, and paste them at the end of the bashrc file, **Fig. 3**. Ctrl O saves the file, and Ctrl X exits the editor. Run `source ~/.bashrc` to reload the file and solidify the changes. To validate the installation, type `nvidia-smi` into the WSL prompt. This command should return the same information as before. Next, type `nvcc -V` into the command prompt. This command should return the CUDA version just installed, **Fig. 4**. It’s unlikely that the CUDA version bundled with the driver is the version necessary for the chosen Tensorflow distribution, so it is expected that these commands will show different CUDA versions. The CUDA version listed from the `nvcc -V` command needs to be the required Tensorflow dependency that the bashrc file points to, and an improperly installed CUDA distribution will result in a failure to properly use GPU memory which can result in CPU bottlenecks, see 5.2.4 or 5.3.2.

```

1 N/A N/A 26 G /Xwayland
satkaylegianstarkeykitty9507@TF-250717213019:~$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Jun_6_02:18:23_PDT_2024
Cuda compilation tools, release 12.5, V12.5.82
Build cuda_12.5.r12.5/compiler.34385749_0
satkaylegianstarkeykitty9507@TF-250717213019:~$

```

Figure 4: CUDA installations can be validated with the `nvcc -V` command which should return the version that the `bashrc` file points to.

```

cuda_vdpau_interop
cudalibxt.h
cudart_platform.h
cudnn.h
cufft.h
cufftXt.h
cufftw.h
cufile.h

```

Figure 5: After a proper cuDNN installation, `cudnn.h` will appear in the `CUDA-v.v/include` folder.

4.5 cuDNN

cuDNN stands for CUDA Deep Neural Network library. Linux distro-specific installers exist as do directions from the cuDNN archive, <https://developer.nvidia.com/rdp/cudnn-archive>, but the most fool-proof installation method is to download the distribution-indifferent tarball file that matches the Tensorflow dependency and manually transfer the files contained within to the proper folders with the following steps:

1. `$ cd <the directory containing the .tar file>`
2. `$ tar -xf <the name of the .tar file>`
3. `$ sudo cp <the name of the .tar file without the .tar extension>/include/cudnn*.h /usr/local/cuda-v.v/include/`
4. `$ sudo cp <the name of the .tar file without the .tar extension>/lib/libcudnn* /usr/local/cuda-v.v/lib64/`

In line 2., the `xf` flag is necessary to properly unpack the tarball file; lines 3. and 4. copy the required files to the `include` and `lib64` subfolders of the CUDA folder that the `bashrc` file points to. Navigating to these folders and confirming that the `cudnn.h` and `libcudnn` files transferred and reflect the desired distribution validates the cuDNN installation, **Fig.5, Fig. 6**. For the authors, this means that a `cudnn.h` file can be found in our `CUDA-12.5/include` folder and `libcudnn` files marked with 9.3.0 are in our `CUDA-12.5/lib64` folder.

```

libcudart.so.12.5.82
libcudart_static.a
libcudnn.so
libcudnn.so.9
libcudnn.so.9.3.0
libcudnn_adv.so
libcudnn_adv.so.9
libcudnn_adv.so.9.3.0
libcudnn_adv_static.a
libcudnn_adv_static_v9.a
libcudnn_cnn.so

```

Figure 6: Validation of the cuDNN installation. A long list of libcudnn files marked with the desired version number must be present in the lib64 folder of the CUDA-v.v folder.

4.6 IDE

The choice of IDE is highly personal. The authors installed the free Pycharm distribution using `sudo snap install pycharm-community --classic` then set the base WSL Python distribution as the interpreter. Regardless of which IDE is preferred, it's best to install it through WSL since Tensorflow versions newer than 2.11 must be installed through WSL for GPU support. The install path, usually `/snap/bin`, will need to be added to the system path in the `bashrc` file in the same way that the CUDA path was added, see the first *export* line in **Fig. 3**; specifics on the expected installation path can be found by reading the IDE's command line installation instructions.

4.7 Tensorflow

From within the chosen IDE, open a Linux terminal and ensure that `pip` is installed and updated with `pip install pip` and `pip install --update pip`. Tensorflow with GPU support can then be installed with the command `pip install tensorflow[and-cuda]`. This installs the newest Tensorflow distribution. If impatient or struggling with the CUDA and cuDNN installations, this command will install all required Tensorflow files and CUDA dependencies, however the CUDA files are not installed into the expected `/usr/local` path so the `nvcc -V` command will return nothing. While this method works, the ideal installation is to first install CUDA and cuDNN through WSL so that the installations can be explicitly verified. If the previous CUDA and cuDNN installations were successful and are the proper dependencies for the given Tensorflow version, the `pip install tensorflow` command will report that all CUDA requirements are already satisfied, and only Tensorflow-specific files will be installed, **Fig. 7**. To install an archived Tensorflow distribution, use `pip install tensorflow[and-cuda]==v.v`.

5 Troubleshooting

This section aims to address common, and not so common, errors encountered while setting up a GPU-enabled Tensorflow training system. We will reiterate our comment from **1** that not all directions will work to solve the same

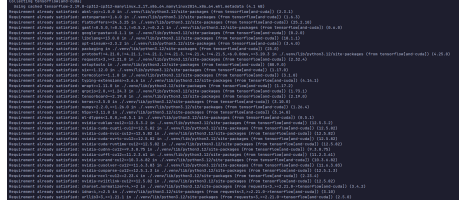


Figure 7: If CUDA and cuDNN are successfully installed prior to installing Tensorflow, all CUDA requirements will be already satisfied.

problem for everyone. The solutions to installation and communication problems are environment specific and sometimes fixes arise almost like magic after the nth attempt. Trouble with physical hardware installations are more rare than CPU/GPU usage bottlenecks caused by faulty communication after apparently successful installations. We begin the troubleshooting section with the few commonly encountered hardware problems in **5.1** then continue to software installation and communication problems in sections **5.2** and **5.3**.

The troubleshooting solutions that follow come from the author’s experiences and represent a well-rounded repository of potential problems. Many attempts at solutions were tried when installations failed or broke; all of the attempts are not documented here, rather we chose to highlight the numerous problems that may arise and the solutions that were successful for us. Knowing that these are not blanket solutions, we provide detailed explanations along with our solutions to guide the reader in further troubleshooting.

5.1 Hardware Installations

5.1.1 GPU

1. Windows Task Manager doesn’t see the newly installed GPU.

This failure mode has two causes: either the GPU is improperly installed or the hardware is faulty.

1. Unplug the system and confirm that the new card is secure and fully plugged in. Restart the PC and check Task Manager again.
2. If 1. fails, test the GPU on another system or another GPU on the given system if possible.
3. If 2. fails, the GPU is most likely faulty. Exchange it under warranty and try the GPU installation again.

2. The PC turns on and shuts off or won’t boot after GPU installation.

This failure mode is very unlikely. It may be caused by charging within the PC during the GPU installation which overloads the system’s capacitors or the hardware may be faulty.

1. Unplug the system for a minimum of 24 hours to allow all capacitors to discharge. If the system still struggles to remain booted after 24 hours the capacitors need to continue discharging. Leave the PC unplugged and repeat boot attempts over the next few days.
2. If the system still won't boot or won't remain booted, remove the GPU and attempt to start the machine. If the system boots the GPU is bad; if it doesn't boot other PC hardware is faulty.

5.1.2 RAM

1. If the system doesn't boot after RAM installation, see **5.1.1.2**.

5.2 Software Installation

5.2.1 Drivers

1. *nvidia-smi* returns nothing or a subset of installed GPUs.

This failure mode has three possible causes: the PC wasn't rebooted after the installation and therefore doesn't see the new driver, the downloaded installer is corrupt, or the new driver is incorrect for the GPU and PC environment.

1. Restart the computer using the Windows Restart button and check the output of *nvidia-smi* after the computer is restarted.
2. If *nvidia-smi* doesn't output the expected information after the restart, it's possible that the installation wasn't successful. Attempt a reinstallation using the same downloaded driver by navigating to Add and Remove Apps using the Windows Search bar, uninstalling all Nvidia programs, then navigating to
C:/Program Files,
C:/Program Files(x86),
C:/User/<User Name>/Local, and
C:/User/<User Name>/Roaming
and removing all Nvidia folders that the system will allow. If the PC says that a folder can't be removed because it's open although it obviously isn't, it's because a default Nvidia graphics driver is using the folder. Leave any folders that can't be deleted. Once all Nvidia files have been cleared from the system they can be, reinstall the new driver with the Windows executable and watch the installation to confirm that there are no errors. Reboot the system and try the *nvidia-smi* command again. If this fails, delete then re-download the driver executable, remove all Nvidia files, and repeat the installation.
3. If *nvidia-smi* still doesn't work as expected, double check the system and graphics card information and ensure that the driver is appropriate for the hardware. Follow the uninstall steps from **5.2.1.2** if the driver is incorrect, download and install the appropriate driver,

restart the PC, and type *nvidia-smi*. Note that older drivers may not be supported by newer GPUs even if they are correct for the card and operating system. If the driver is appropriate for the environment and all troubleshooting steps fail, the driver is too old for the card to support. Wipe all Nvidia folders from the Windows Native system, download a newer driver, and try again.

5.2.2 WSL

1. Typing *wsl* in a command prompt displays an error message that there is no installed distribution and therefore WSL cannot open.

This failure mode occurs if a Linux distribution isn't specified when installing WSL.

1. Reinstall WSL. Be sure to follow the directions in **4.2** and ensure that a distribution name is explicitly specified upon installation.
2. The *wsl* command fails to open the WSL prompt. There may or may not be error messages.

This occurs when the WSL executable installation fails.

1. Refer to **4.2**, and be sure to follow the installation instructions using *wsl.exe*. Don't perform the install without the .exe file extension.

5.2.3 Python

1. *pythonv -V* only returns the base WSL interpreter and no files exist for the additional Python distribution.

It is very rare that a pip installation of Python will fail, but there are countless reasons why it might.

1. Type *sudo apt purge pythonv**, where v is the newly installed version, then *sudo apt autoremove* and *sudo apt autoclean* to wipe all traces of the second Python distribution. Attempt the pip install again.
2. If the system dislikes pip installing Python, try *sudo apt install pythonv-pip*.
3. If pip and sudo don't work, or if it's expected that there will be multiple Python versions on the system which will require version control, use a Miniconda environment. Miniconda creates a base environment that contains the version of Python it was built with while allowing for the creation of additional environments built with other Python versions. Miniconda can be installed with
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-Windows-x86_64.exe -o .\miniconda.exe [4].

2. Installing Python packages into the IDE after the new Python version has been set as the interpreter fails or `pip install <some Python package>` says that the package isn't available.

There is one root cause for this failure mode: the new Python distribution hasn't been added to the system path.

1. Add the new Python distribution to the system path in the `bashrc` file.

5.2.4 CUDA

1. `nvcc -V` returns nothing or a different CUDA version than was just installed.

This may happen if the CUDA path has not been added to the system's environmental variables or if the CUDA installation was corrupted.

1. Use a text editor to open the `bashrc` file. Make sure the path is directed at the new CUDA installation. If it points to an old installation, remove the path variables and add those associated with the new installation. If the path variables were missing, add them according to **Fig. 3**.
2. If the `bashrc` files was pointed to the proper path, navigate to `/usr/local/` and confirm that a `CUDA` and a `CUDA-v.v` folder exist. The `CUDA-v.v` folder should match the version just installed. If either of these folders is absent or empty the installation was corrupted, and CUDA will need to be reinstalled. Before reinstalling, wipe all existing CUDA files from the system using the following commands:
 1. `$ sudo apt --purge remove "*cublas*" "cuda*" "nsight"`
 2. `$ sudo apt --purge remove "*nvidia*"`
 3. `$ sudo rm -rf /usr/local/cuda*`
 4. `$ sudo apt autoremove`
 5. `$ sudo apt autoclean`

Once all CUDA files have been removed, retry the installation.

3. If the installation fails again, it's most likely because the archived distribution has a corrupted installer. In this case, attempting the install using the distro-independent tarball file may work, but it's best to try a newer CUDA version if possible.

5.2.5 cuDNN

1. The `cuda.h` and `libcudnn` files aren't present in the `CUDA-v.v` folder. This may happen if the files weren't explicitly transferred from the CUDA folder or if the installer download is corrupt.

1. Reattempt the cuDNN installations following the steps in **4.5**.
2. If the files still aren't present, it is advisable to try the deb local installer for the desired cuDNN version.
3. If both the tarball and deb local installers fail, the archived cuDNN file is corrupt. Choose another cuDNN distribution; be sure to check Tensorflow and CUDA compatibility with the new cuDNN version, and follow the steps in **5.2.4.2** to clear all previous CUDA, cuDNN, and Nvidia files before attempting the CUDA/cuDNN installation with a different cuDNN distribution.

5.2.6 IDE

1. Attempting to open the IDE returns a message that the IDE command exists in a different directory or returns nothing.

This failure mode occurs when the IDE path is not present in the bashrc file.

1. Add the IDE path to the bashrc file according to **Fig. 3**.
2. If the IDE still can't be found, the installation is corrupt. Use *sudo apt purge* and *sudo apt autoclean* to erase all existing IDE files and attempt the installation again.
2. The IDE freezes while loading data or training models.

This isn't a failure mode of the IDE but rather one of failed communication between the driver/CUDA/cuDNN/Tensorflow and will be discussed in **5.3**.

5.2.7 Tensorflow

If Tensorflow imports, the installation was successful. Data loading and training errors are the result of CUDA/cuDNN/Tensorflow distribution mismatches, failed CUDA/cuDNN installs, or out of data drivers and will be addressed in **5.3**.

It is worth noting that, upon import, Tensorflow is verbose and will potentially print error messages regarding failed cublas or other CUDA package registration because the package is already registered. This is a benign message. Tensorflow may also print a message referencing CPU enablement despite having just installed Tensorflow with GPU support; this message occurs because the default Tensorflow build is for CPU. It is a benign message. Tensorflow can be silenced by only allowing fatal errors to print, **Fig. 8**. To silence Tensorflow, add the following lines when importing at the beginning of the script:

```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=FutureWarning)
```

```
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
warnings.filterwarnings("ignore", category=FutureWarning)
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
```

Figure 8: These five lines silence all non-essential Tensorflow warnings.

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'.
```

5.3 Everything Else

If the issue encountered isn't related to a hardware or a software installation but instead is related to data loading or model training, this is the right section. These problems typically stem from failed communication between drivers/CUDA/cuDNN/Tensorflow, and because the failed communication can come from any one or any combination of programs these issues are the most challenging to troubleshoot.

5.3.1 Data Loading

1. The IDE freezes or an out of memory (OOM) error is encountered while loading or pre-processing data.

Both of these failure modes have the same root causes: either the CUDA/cuDNN installations failed or are the incorrect dependencies for the installed Tensorflow distribution or the data loading portion of the script needs to be optimized.

1. Follow the installation validation steps in 4 to ensure that all CUDA and cuDNN files are present and that the installed driver is supported by the installed GPU. Confirm that these versions are the required dependencies for the installed Tensorflow version.
2. If all the Tensorflow dependencies are correct and properly installed, confirm that Tensorflow can see the installed GPU by opening a Python terminal in the given project, importing Tensorflow, and typing `tf.config.experimental.list_physical_devices('GPU')`; this command checks which GPUs Tensorflow can see.
3. If Tensorflow is correctly built, the freezes or OOM errors are the result of unoptimized data loading. Through Keras, Tensorflow can access the Utility API which provides already optimized data loading functions. For example, large image datasets can be loaded using the `tf.keras.utils.image_dataset_from_directory` API, **Fig. 9**; see https://keras.io/api/data_loading/ for details on function args and kwargs. Other than using the Keras data utility to load data, it is recommended to use prefetched, AUTOTUNED, and cached datasets,

```
#### define functions ####
def creating_datasets():
    print('creating training dataset:')
    img_training = tf.keras.utils.image_dataset_from_directory(sorted_path,
                                                              color_mode='grayscale', seed=1, validation_split=0.2,
                                                              subset='training', labels='inferred',
                                                              label_mode='categorical',
                                                              shuffle=True, image_size=(height, width),
                                                              batch_size=batch) #shuffle=True

    print('creating validation dataset:')
    img_val = tf.keras.utils.image_dataset_from_directory(sorted_path,
                                                         color_mode='grayscale', seed=1, validation_split=0.2,
                                                         subset='validation', labels='inferred',
                                                         label_mode='categorical',
                                                         shuffle=True, image_size=(height, width), batch_size=batch
                                                         )
```

Figure 9: An example of how to load training and validation image data with the Keras Utility API. Note that test data can be loaded in the same manner by not specifying the validation split or subset flags.

```
print('optimizing datasets:')
AUTOTUNE = tf.data.AUTOTUNE

img_training = img_training.cache().shuffle(img_training.cardinality()).prefetch(buffer_size=AUTOTUNE)
img_val = img_val.cache().shuffle(img_val.cardinality()).prefetch(buffer_size=AUTOTUNE)
img_test = img_test.cache().shuffle(img_test.cardinality()).prefetch(buffer_size=AUTOTUNE)
```

Figure 10: An example of optimizing keras.utils loaded data with prefetch, AUTOTUNE, and cache functions.

Fig.10. The Tensorflow prefetch function optimizes the data loading pipeline in conjunction with model training [5]; AUTOTUNE optimizes data loading across the CPU [6], and caching the dataset ensures that essential data-specific functions are kept in memory and only run once during training. For example, using a cached dataset ensures that data will only be loaded at the beginning of training and not at the beginning of each epoch [7].

4. If the IDE continues to freeze or OOM errors persist, try: 1) using a smaller batch size during training so that less data is loaded at a single time, 2) resizing the data, 3) changing the data format, 4) reducing the model complexity.

5.3.2 Model Training Slowdowns

1. The IDE freezes, crashes, or training suddenly takes longer than it used to without major architecture/dataset changes.

These three failure modes all stem from failed driver/CUDA/cuDNN/Tensorflow communication.

1. To confirm that failed communication is the cause of the given failure mode, open Windows Task Manager and click on Performance. Watch the GPU and CPU while training. Failed communication means that the GPU isn't visible or that it is not properly used during training, and it may look like **Fig. 11** where the GPU shared memory is used but training occurs on the CPU instead of using GPU-dedicated memory. This means that the GPU is visible to Tensorflow but CUDA and cuDNN aren't properly configured.

Another way to check that these failure modes are the result of failed communication is to open the Windows Reliability Tracker

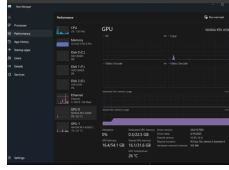


Figure 11: The driver is correctly installed but CUDA and cuDNN are incorrectly configured because training uses the GPU shared memory then moves to the CPU instead of using the dedicated GPU memory.

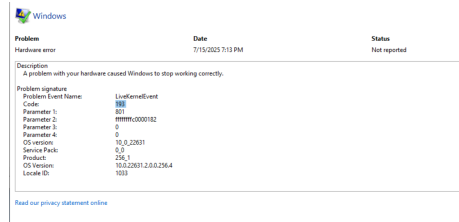


Figure 12: A Windows KillKernel error 193 like this implies driver or hardware problems.

using the Search bar. Look for critical errors which appear as a red circle with a white x in the middle. If a critical error code 193 or 141, **Fig. 12**, appears and Task Manager looks like **Fig. 11**, or if there are no error codes but training looks like **Fig. 11**, the problem is related to Nvidia and CUDA; move to step 2. If error codes are present but training doesn't look like **Fig. 11**, the problem is related to the system's hardware, see **Note 2.** at the end of this section.

2. If Nvidia/CUDA communication is the expected root cause, check the CUDA-v.v folder and confirm that the expected cuDNN files are still present then query `nvcc -V` to ensure the bashrc file still points to the expected CUDA version. It is very unlikely files and paths were removed, but if a Windows update recently occurred it's possible. If the cuDNN files are missing or the bashrc file no longer points to the expected distribution, follow the steps in **4.5** for cuDNN and in **4.4** for CUDA.
3. If the outputs from 2. are as expected, open a Linux terminal from within the IDE and type `pip install nvidia-cudnn-cu*` where the final v is for the installed CUDA version. For the authors this command looks like `pip install nvidia-cudnn-cu12`. This command installs the latest cuDNN version into the Python environment; this will not change any files in the CUDA-v.v folder. Attempt to train.
4. If the error persists, uninstall and reinstall the given Tensorflow distribution. Attempt to train.

5. If this doesn't fix the training errors, use *pip uninstall* to remove the cuDNN version installed in 3., and navigate to <https://www.nvidia.com/en-us/drivers/>. Find the newest driver in the currently installed series; **DON'T CHANGE DRIVER SERIES EVEN IF MORE THAN ONE SERIES IS APPROPRIATE FOR THE GIVEN GPU**. There will come a time when changing series is an appropriate troubleshooting step, but not yet. This is the troubleshooting stage where changing as little as possible is preferred. For example, driver series 576 and 573 are both appropriate for an A5000 card, so if driver 576.02 is installed, the upgrade is to driver 576.48, not driver 573.48. Download the new driver and uninstall the old driver using Windows Add/Remove programs. Navigate to Program Files, Program Files(x86), and Users and delete all Nvidia folders; no CUDA folders should exist in any of the aforementioned locations. Once all previous Nvidia files are cleared from the system, reboot, and install the new driver following the steps in 4.1. After rebooting the system attempt to train.
6. If the error persists keep the updated driver. It's time to wipe and reinstall CUDA. Follow the steps in 5.2.4.2 to remove all CUDA and cuDNN files then reinstall following 4.4 and 4.5. After a successful installation, pip uninstall and pip install Tensorflow with the *tensorflow[and-cuda]* command. Validate the installation following the procedures outlined in **Software Installation and Validation**. Attempt to train. If this clean installation with the updated driver fails, it's time to update CUDA, cuDNN, and Tensorflow.
7. Again, follow the steps in 5.2.4.2 to remove all CUDA and cuDNN files. Pip uninstall Tensorflow. Open <https://www.tensorflow.org/install/source#gpu> and choose the latest Tensorflow distribution; if currently using the newest distribution roll back one. Collect the required dependencies for the new Tensorflow distribution, and follow the steps in **Software Installation and Validation** to install and validate the new CUDA, cuDNN, and Tensorflow builds. Always be sure to add paths to the bashrc file and confirm that the cuDNN files are present in the CUDA-v.v folder. Install the new Tensorflow version using a Linux terminal in the IDE, then try training.
8. If training is still slow or the IDE freezes or kills itself, it's time to change driver series. Follow the steps outlined to remove and reinstall all things CUDA and Tensorflow so that the newest Tensorflow build is installed with the proper CUDA and cuDNN dependencies. Follow the steps previously outlined to uninstall the current driver and remove all subsequent Nvidia folders. From <https://www.nvidia.com/en-us/drivers/> choose the newest driver in the next highest compatible series. Install the driver and attempt to train.
9. If 8. fails, repeat the driver uninstall and reinstall, choosing the next oldest driver in each series.



Figure 13: High CPU usage by WSL with no model training or open IDE suggests a WSL memory leak.

It is unlikely that troubleshooting will reach step 9., although if it does the pursuit isn't hopeless. It takes time, patience, and clean installations when attempting to use GPU-enabled Tensorflow training in Windows. If possible, disable Windows updates to minimize communication failures, although experience has shown that CUDA and Tensorflow communication can break suddenly with no inherent system changes, and it has shown that refreshing the cuDNN installation and/or updating the driver is often sufficient to restore optimized training. With that, we will reiterate that the cuDNN files must be present in the expected folders; cuDNN is crucial for proper GPU and CPU communication, and even when Tensorflow sees the GPUs and all dependencies appear happy and successfully installed, failing to have the cuDNN files in the right locations will cause severe training slowdowns and IDE kill errors.

5.4 Notes

- Note 1:** While rare, changing the training environment can cause variations when retraining previously stable models. At best we conclude that updated Tensorflow versions may have differences in rounding that become significant while training or that updated cuDNN algorithms change the communication between CPU and GPU which changes the treatment of floating point values and has a noticeable effect on training. As mentioned early on, our unstable system became more unstable over time, and part of that instability included significant variances in training after restoring communication, namely that our previous models were not reproducible because previously successful architectures had drastically different learning with the restored system.
- Note 2:** If training uses dedicated GPU memory but the IDE freezes and there are Windows KillKernel errors in the Reliability Tracker, see **5.1.1** to troubleshoot the GPU. If the GPU isn't faulty, it's time to reimagine the PC, and if this fails, the fault is with native system hardware.
- Note 3:** Check for memory leaks. For example, **Fig. 13** shows a memory leak in WSL which retains high CPU usage after training and Pycharm have been killed. Acknowledgment of this memory leak was one key factor in the author's decision to move away from Windows and WSL to a Linux-only environment.
- Note 4:** A good sanity check is to confirm that Tensorflow sees the GPU, **Fig. 14**. This can be done by importing Tensorflow in a Python terminal then querying with `tf.config.experimental.list_physical_devices('GPU')`.

```

2019-10-09 16:16:15.870000: I tensorflow/core/platform/cpu_feature_guard.cc:140] Your TensorFlow binary is optimized
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AVX_Tile AVX_DQBF AVX
Backend flag is interactive backend. Turning interactive mode on.
In [3]: physical_devices
Out[3]:
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU'),
 PhysicalDevice(name='/physical_device:GPU:1', device_type='GPU')]
In [4]:

```

Figure 14: If the drivers and GPU are properly installed and Tensorflow can see the GPU, the physical devices command will print a list of all devices that Tensorflow sees for training. Here, only the available GPUs are printed, but by changing ‘GPU’ to ‘CPU’ we can print all available CPUs.

```

os.environ['TF_FORCE_GPU_ALLOW_GROWTH'] = 'true'
# os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
import tensorflow as tf
physical_devices = tf.config.experimental.list_physical_devices('GPU')
for gpu in physical_devices:
    tf.config.experimental.set_memory_growth(gpu, True)

```

Figure 15: Adding these lines during imports ensures that data is loaded into GPU memory as a step function and not all at once.

Note 5: Tensorflow’s innate nature is to load everything it can at once which may overload the system and cause crashes. Visually, when watching Task Manager, this looks like a steep jump in GPU shared and dedicated memory. This can be mitigated by forcing Tensorflow to gradually load memory. Visually, GPU shared and dedicated memory usage will increase in a step function. To force memory growth place `os.environ['TF_FORCE_GPU_ALLOW_GROWTH'] = 'true'` before importing Tensorflow and `for gpu in physical_devices: tf.config.experimental.set_memory_growth(gpu, True)` where `physical_devices` is the output of `tf.config.experimental.list_physical_devices('GPU')` after importing Tensorflow, **Fig. 15**.

6 Final Thoughts

Building and training machine learning models requires enough RAM and storage to hold data and a graphics card for quick computation. The graphics card requires a driver to communicate with the system and CUDA and cuDNN to optimize GPU/CPU communication. Identifying proper dependencies for the desired Tensorflow version is critical as is ensuring that the necessary cuDNN files are in the CUDA-v.v folder, but even with clean installations, optimized training can be challenging as commands work from Python and the IDE through Linux and WSL. In this guide we have outlined the necessary hardware and software components for model training and provided in-depth installation and troubleshooting instructions. This guide is as complete as any Windows and Tensorflow troubleshooting guide can be, but it’s guaranteed that new systems

and environments will encounter new problems not mentioned here, and it's guaranteed that environmental specifications will make some troubleshooting or installation step fail for some reader despite being successful for the authors. This is the nature of what we do. At best the authors hope that all installations are forever successful; at worst, we hope we have provided well-rounded and complete knowledge so that, as new problems arise, troubleshooting is less of a shot in the dark and more of a calculated plan.

7 Citations

1. "Microsoft:- WSL – Supported Distros (2025 – March)," Learning in the Open, <https://learningintheopen.org/2025/03/13/microsoft-wsl-supported-distros-2025-march/>.
2. "How to install Linux on Windows with WSL," Microsoft Ignite, <https://learn.microsoft.com/en-us/windows/wsl/install>.
3. "Introduction to CUDA Programming," Geeks for Geeks, <https://www.geeksforgeeks.org/electronics-engineering/introduction-to-cuda-programming/>.
4. "Installing Miniconda," Anaconda, <https://www.anaconda.com/docs/getting-started/miniconda/install>.
5. "Better performance with the tf.data," Tensorflow Core, API https://www.tensorflow.org/guide/data_performance.
6. "Tensorflow tf.data.AUTOTUNE," Stack Exchange, Tensorflow <https://stackoverflow.com/questions/56613155/tensorflow-tf-data-autotune>.
7. <https://stackoverflow.com/questions/59228816/what-do-the-tensorflow-datasets-functions-cache-and-prefetch-do>